TE Artificial Intelligence and Soft Computing

Mini Project Report

On

HackMate AI: An Intelligent AI-Based Hackathon Discovery and Recommendation System

Submitted in partial fulfillment of the requirement of
University of Mumbai for the Degree of

Bachelor of Engineering

in

Artificial Intelligence and Data Science.

Submitted By

Divya Sharad Dhanawade

Supervisor

Prof Nirosha Uppu

Department of Artificial Intelligence and Data Science.

Smt. Indira Gandhi College of Engineering, Ghansoli – 400701

An Autonomous Institute

Academic Year 2026 – 27

Department of Artificial Intelligence and Data Science.

SMT. INDIRA GANDHI COLLEGE OF ENGINEERING

GHANSOLI – 400701 (An Autonomous Institute)

CERTIFICATE

This is to certify that the requirements for the TE Mini Project report entitled '**HackMate AI: An Intelligent AI-Based Hackathon Discovery and Recommendation System**' have been successfully completed by the following students:

Name	**Roll No.**
Divya Dhanawade	18

in partial fulfillment of Bachelor of Engineering in Department of Artificial Intelligence and Data Science, Smt. Indira Gandhi College of Engineering, Ghansoli 401701,An Autonomous Institute during the Academic Year 2026 – 2027.

Prof .Nirosha Uppu
Supervisor

Dr. Shankar M. Patil
Head of Department

Dr. Sunil Chavan
Principal

Department of Artificial Intelligence and Data Science

SMT. INDIRA GANDHI COLLEGE OF ENGINEERING

GHANSOLI – 400701 (An Autonomous Institute)

REPORT APPROVAL

This TE Mini Project report entitled "**HackMate AI: An Intelligent AI-Based Hackathon Discovery and Recommendation System**" by **Divya Sharad Dhanawade** is approved for the degree of **Bachelor of Engineering** Department of Artificial Intelligence and Data Science**, Smt. Indira Gandhi College of Engineering, Ghansoli.** (An Autonomous Institute)

Examiners:

1. _____

2. _____

Supervisors:

1. _____

2. _____

Chairman:

1. _____

Date:

Place:

Declaration

We declare that this written submission for TE Mini Project Report Declaration entitled "**HackMate AI: An Intelligent AI-Based Hackathon Discovery and Recommendation System**" represents our ideas in our own words and where others' ideas or words have been included. We have adequately cited and referenced the original sources. We also declared that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any ideas / data / fact / source in our submission. We understand that any violation of the above will cause disciplinary action by the institute and also evoke penal action from the sources which have thus not been properly cited or from whom paper permission has not been taken when needed.

Project Group Members:

Student Name & Sign: _____

Date:

Place:

Table of Contents

Abstract..... i

List of Figures..... ii

List of Tables..... iii

1. Introduction..... 1

1.1 Fundamentals..... 1

1.2 Objectives..... 2

1.3 Scope..... 3

1.4 Outline……..... 3

2. Literature Survey..... 4

2.1 Introduction……. ……..... 4

2.2 Literature Review …….... 5

2.3 Summary of Literature Survey.……..... 7

3. Proposed System…… 8

3.1 Overview…….... 8

3.1.1 Existing System Architecture……. 10

3.1.2 Proposed System Architecture…… 11

3.2.3 Implementation Details ……..... 12

3.2.1 Algorithm / Tools/ Techniques 12

3.2.2 UseCase/Activity Diagram 13

3.2.3 Dataset and Input Details ……..... 13

3.2.4 Hardware and Software Specifications 14

4. Performance and Results 15

4.1 Output and Graphs…… 16

4.2 Comparative Results 20

5. Applications 21

5.1 Social

6. Conclusion and Future Scope 22

References.....23

Acknowledgement…….....24

Abstract

Hackathons provide an important platform for students and developers to showcase their technical skills, solve real-world problems, and develop innovative solutions. However, finding suitable hackathons according to individual interests, skills, experience levels, and participation preferences can be challenging. Various techniques, such as keyword-based search, manual browsing, filtering systems, recommendation algorithms, and machine learning, are used for hackathon discovery. Traditional search methods often provide limited personalization and require users to explore multiple platforms. To overcome these limitations, **HackMate AI: An Intelligent AI-Based Hackathon Discovery and Recommendation System** is developed using Python and Streamlit. It provides an interactive platform for discovering hackathons through search and filtering features based on participation preferences.

The system incorporates ranked search, a skill-gap advisor, learning-path recommendations, a neural-style problem classifier, a team skill-complement matcher, and a fuzzy difficulty analyzer. These features help users explore relevant opportunities, identify missing skills, plan their learning, categorize problem statements, and understand team skill requirements. The system aims to simplify hackathon discovery and support students in making informed decisions about participation and preparation. HackMate AI provides a centralized and user-friendly platform that encourages collaboration, skill development, and innovation among students and developers.

List of Figures

Figure No.	Title of the Figure	Page No.
Fig. 1.1	HackMate-Ai System Overview	3
Fig. 2.1	HackMate-Ai Recommendation Architecture	5
Fig. 3.1	Classification of AI and Soft Computing Techniques in HackMate-Ai	6
Fig. 3.2	Existing System Architecture of HackMate-Ai	8
Fig. 3.3	Proposed System Architecture of HackMate-Ai	9
Fig. 4.1	HackMate-Ai Dashboard for Hackathon Search and Recommendations	14
Fig. 4.2	Detailed Hackathon Listings with Skill-Match and Difficulty Analysis	15
Fig. 4.3	Neural Network Problem Classification in HackMate-Ai	15
Fig. 4.4	Team Skill-Complement Matching in HackMate-Ai	16
Fig. 4.5	Hackathon Learning Path Advisor in HackMate-Ai	17
Fig. 4.6	Fuzzy Difficulty Analysis in HackMate-Ai	18

List of Tables

List of Tables

Table No.	Title of the Table	Page No.
Table 1.1	Mapping of AI Techniques and Subsystems in HackMate-Ai	2
Table 2.1	Summary of Literature Survey	5
Table 3.1	Sample Dataset Used for Hackathon Discovery and Recommendation	11
Table 3.2	Hardware Details	12
Table 3.3	Software Details	12
Table 4.1	Feature Verification of HackMate-Ai	16
Table 4.2	Comparison of Existing and Proposed Systems	17

Chapter 1
Introduction

1.1 Fundamentals

Hackathons provide an important platform for students, developers, and technology enthusiasts to collaborate, solve real-world problems, and develop innovative solutions. However, finding suitable hackathons based on individual skills, interests, experience levels, and participation preferences can be challenging. Traditional discovery methods often depend on manual browsing and basic keyword searches, making the process time-consuming. Students may also face difficulties identifying missing skills, selecting learning resources, and finding teammates with complementary technical abilities.

HackMate-Ai is a web-based platform developed using Python and Streamlit to simplify hackathon discovery and preparation. It provides search, filtering, and ranked results to help users find relevant opportunities. The system also includes a skill-gap advisor, learning-path advisor, neural-style problem classifier, fuzzy difficulty analyzer, and team skill-complement matcher.

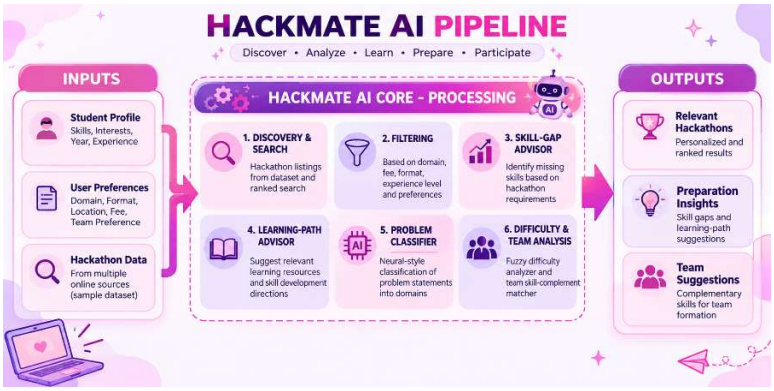


Fig 1.1: HackMate-Ai

Figure 1.1 illustrates the overall pipeline of HackMate-Ai, showing how user inputs are processed to provide relevant hackathon recommendations and preparation insights.

1.2 Objectives

Table 1.1 summarizes the core AI-inspired techniques integrated within the HackMate-Ai architecture along with their respective functional roles.

Subsystem / Experiment	AISC Technique	Primary Objective
Hackathon Search	BFS, DFS	Explore hackathon listings
Intelligent Search	Greedy Search, A*	Find relevant opportunities
Optimization	Hill Climbing, Simulated Annealing	Improve search solutions
Skill-Gap Analysis	Rule-Based Reasoning	Identify missing skills
Problem Classification	Neural Network Concepts	Classify problem statements
Difficulty Analysis	Fuzzy Logic	Estimate problem difficulty

Table 1.1: Mapping of AI Techniques and Subsystems in HackMate-Ai

The objective of this work is as follows:

1. To implement BFS and DFS algorithms for searching hackathon opportunities.
2. To apply A* search, Greedy Search, Hill Climbing, and Simulated Annealing techniques for intelligent search and optimization.
3. To integrate neural networks, fuzzy logic, and rule-based reasoning for problem classification, difficulty analysis, skill-gap identification, and team matching.

1.3 Scope

HackMate-Ai is scoped as an intelligent hackathon discovery and recommendation platform designed for students, developers, and hackathon participants. The deliverables include a Python-based Streamlit web application integrating AI and Soft Computing techniques. Key features include BFS and DFS for search, Greedy Search and A* for intelligent search, Hill Climbing and Simulated Annealing for optimization, rule-based reasoning for skill-gap analysis, neural network concepts for problem classification, fuzzy logic for difficulty analysis, and team skill-complement matching. The platform also provides hackathon filtering, ranked search results, and learning-path recommendations to support opportunity discovery and preparation. The project focuses on academic use, skill development, and helping students identify suitable hackathons through a centralized and user-friendly interface.

1.4 Outline

The report is organized as follows: Chapter 1 introduces HackMate-Ai, explaining the fundamental AI and Soft Computing techniques used, along with the project objectives and scope. Chapter 2 presents the literature survey of existing hackathon discovery and recommendation systems, highlighting their advantages and limitations. Chapter 3 describes the proposed system architecture, algorithms, AI modules, dataset, and hardware and software requirements. Chapter 4 discusses the implementation, testing, and results of the developed system. Chapter 5 presents the societal and technical applications of HackMate-Ai. Finally, Chapter 6 summarizes the work and discusses future enhancements.

Chapter 2

Literature Survey

A literature review provides an objective and critical summary of published research related to the selected domain. It helps understand existing techniques, their applications, advantages, and limitations, and identifies opportunities for further development. This chapter reviews Artificial Intelligence and Soft Computing techniques relevant to **HackMate-Ai**, focusing on search algorithms, optimization methods, fuzzy logic, neural networks, and recommendation systems.

A typical literature review consists of the following components:

1. Introduction
2. Main Body
3. Conclusion
4. References

2.1 Search, Optimization, and Recommendation Techniques

Search and optimization techniques are widely used in Artificial Intelligence to explore possible solutions, identify relevant information, and improve decision-making. In HackMate-Ai, these techniques support hackathon discovery, search, ranking, and optimization-related operations. The following subsections describe these methods, their applications, advantages, and limitations.

2.1.1 Uninformed Graph Traversal and State-Space Search

Russell and Norvig [2] describe uninformed search algorithms, particularly Breadth-First Search (BFS) and Depth-First Search (DFS), as fundamental methods for exploring search spaces. BFS explores nodes level by level, whereas DFS explores one branch deeply before backtracking. These algorithms are useful for systematically traversing connected structures and exploring possible solutions.

In HackMate-Ai, BFS and DFS are associated with the search functionality for exploring hackathon opportunities. BFS provides level-wise exploration, while DFS supports depth-wise traversal. The primary advantage of these methods is their systematic exploration of a search space. BFS can find the shortest path in an unweighted graph, whereas DFS generally requires less memory for certain

deep search structures. However, BFS may require substantial memory, and DFS does not guarantee the shortest path. Their effectiveness depends on how the search space is represented.

2.1.2 Heuristic Search and Optimization Techniques

Russell and Norvig [2] discuss informed search methods such as Greedy Best-First Search and A* Search. Greedy Search selects the next state using a heuristic estimate, whereas A* combines the actual cost of reaching a state with the estimated cost of reaching the goal. Hill Climbing improves a current solution by selecting a better neighbouring state. Kirkpatrick et al. [3] introduced Simulated Annealing as an optimization method inspired by the physical annealing process, allowing the search to accept some worse solutions to explore a wider solution space.

In HackMate-Ai, these techniques are relevant to intelligent search and optimization concepts for exploring hackathon opportunities. Their main advantage is that they can guide the search towards promising solutions and improve exploration. However, Greedy Search may miss the optimal solution, Hill Climbing may become trapped in local optima, and Simulated Annealing depends on its cooling schedule. A* can also require considerable memory for large search spaces.

2.2 Soft Computing and Machine Learning Techniques

Soft Computing and Machine Learning techniques help systems handle uncertainty, identify patterns, and support decisions using available information. HackMate-Ai incorporates AI-inspired approaches for problem classification, skill-gap analysis, difficulty estimation, learning recommendations, and team skill matching. The following sections describe the relevant techniques and their advantages and limitations.

2.2.1 Fuzzy Logic and Rule-Based Reasoning

Zadeh [4] introduced Fuzzy Set Theory to represent information through degrees of membership ranging from 0 to 1 rather than using only strict binary values. Fuzzy logic is useful when information cannot be classified accurately using fixed categories. Rule-based reasoning uses predefined conditions and rules to produce decisions from given inputs.

In HackMate-Ai, fuzzy logic is associated with the difficulty analyzer, which estimates the difficulty of hackathon problem statements. Rule-based methods support skill-gap analysis and learning recommendations. The main advantage of fuzzy logic is its ability to represent gradual changes and uncertain information. Rule-based reasoning is comparatively easy to understand and interpret. However, the quality of the results depends on the selected membership functions, rules, and input information.

2.2.2 Neural Networks and Problem Classification

Rosenblatt [5] introduced the Perceptron as a model for information processing and pattern recognition. Neural networks use interconnected computational units to process input features and produce outputs. A Perceptron calculates a weighted combination of inputs and applies an activation function to determine its output.

In HackMate-Ai, a neural-style problem classifier is used to categorize hackathon problem statements into relevant technical domains. However, their accuracy depends on the quality and quantity of training examples, feature selection, and model design. Limited training data may also affect the reliability of the predicted category.

2.2.3 Recommendation and Team Skill Matching

Recommendation systems help users identify relevant items by considering preferences, available information, and matching criteria. Search ranking and filtering can reduce the effort required to explore a large collection of opportunities. Skill-based matching can also compare user abilities with the requirements of a task or team.

In HackMate-Ai, ranked search helps users explore relevant hackathons, while the skill-gap advisor identifies areas that may require improvement Their main advantage is convenience and improved organization of relevant information. However, recommendation quality depends on the completeness of hackathon data, the accuracy of user-provided skills, and the matching criteria used by the system.

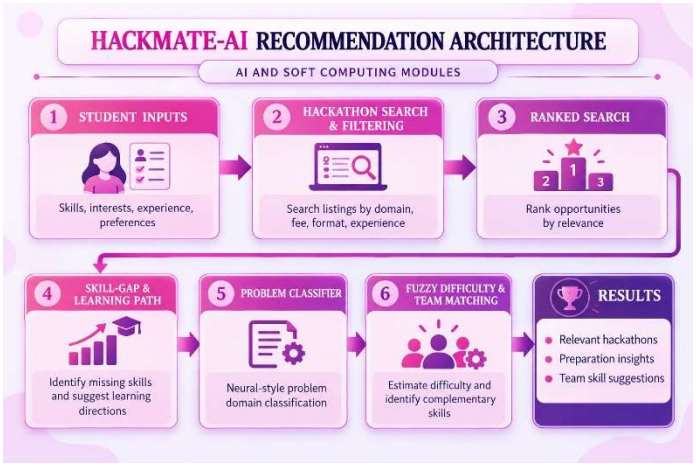


Fig. 2.1: HackMate-Ai Recommendation Architecture

2.3 Summary of Literature Survey

Th A literature review provides a critical summary of published research relevant to the selected topic. The summary of literature related to the AI and Soft Computing techniques used in HackMate-Ai is presented in Table 2.1.

SN	Paper / Author	Advantages and Disadvantages
1.	S. Russell and P. Norvig [2] (BFS, DFS, A and Greedy Search)*	**Advantages**: Provides systematic search and exploration of possible solutions. Heuristic methods help guide the search towards promising results. **Disadvantages**: BFS may require high memory, DFS does not guarantee the shortest path, and heuristic methods depend on the quality of the heuristic.
2.	L. A. Zadeh [4] (Fuzzy Sets and Fuzzy Logic)	**Advantages**: Handles uncertain information using degrees of membership and supports flexible difficulty estimation. **Disadvantages**: Results depend on the selected membership functions and fuzzy rules.
3.	F. Rosenblatt [5] (Perceptron and Neural Networks)	**Advantages**: Supports pattern recognition and classification of hackathon problem statements into technical domains. **Disadvantages**: Classification performance depends on training data, input features, and model design.
4.	S. Kirkpatrick et al. [3] (Simulated Annealing)	**Advantages**: Explores different solutions and may avoid becoming trapped in local optima during optimization. **Disadvantages**: Performance depends on the cooling schedule and parameter selection.

Table 2.1: Summary of Literature Survey

Chapter 3

Proposed System

3.1 Overview

The system overview of **HackMate-Ai** is presented in this section. The classification of the Artificial Intelligence and Soft Computing techniques used in hackathon discovery, optimization, problem classification, and recommendation is shown in Figure 3.1.

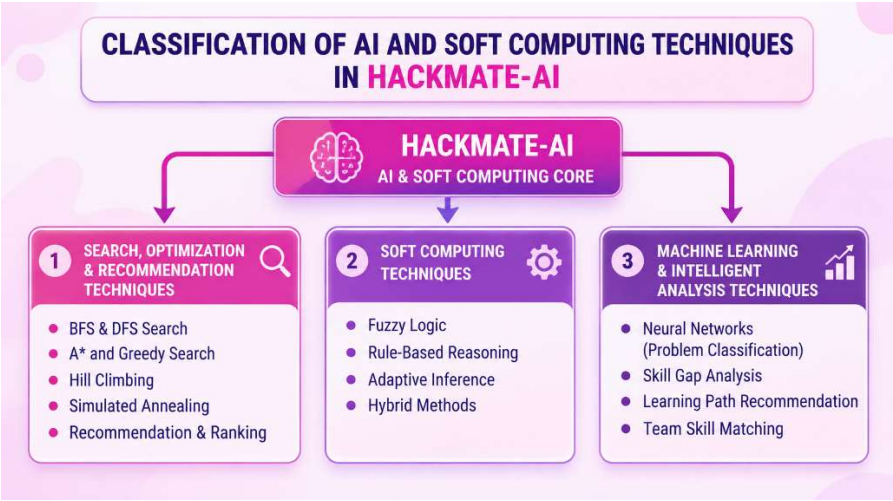


Fig. 3.1 Classification of AI and Soft Computing Techniques in HackMate-Ai

This system helps students and developers discover relevant hackathon opportunities by analyzing their skills, interests, and participation preferences. Based on the challenges of traditional hackathon discovery, recommendation techniques can be classified into the following categories:

Personalized Recommendation: Personalized recommendation adapts suggestions to the individual needs, interests, and preferences of each user. It includes content-based recommendation, collaborative filtering, and knowledge-based recommendation techniques. In HackMate-Ai, user skills, interests, experience levels, participation formats, and fee preferences help users explore suitable opportunities. Ranked search and filtering support hackathon discovery, while skill-gap analysis and learning-path suggestions assist in preparation. The main advantage is that recommendations can be aligned with individual requirements. However, their effectiveness depends on the accuracy of user information and the availability of relevant hackathon data.

Non-Personalized Recommendation: Non-personalized recommendation systems provide similar suggestions to all users without considering individual preferences. These systems generally display popular, highly rated, or commonly selected items. In hackathon discovery, such a system may

present a common list of popular hackathons to every user. Its main advantage is simplicity and ease of implementation. However, it may suggest opportunities that do not match a student's technical skills, interests, or experience level.

Aggregated Opinion Recommendation: Aggregated opinion recommendation uses ratings or feedback from multiple users to calculate an overall score for an item. For example, hackathons can be compared using average participant ratings when reliable feedback is available. The average score is calculated using Equation (2.1):

Average Score = $(R_1 + R_2 + \cdots + R_n) / n$

Content-Based Recommendation Systems: Content-based recommendation systems analyze item descriptions and compare their characteristics with a user's interests or profile. The recommendation process involves identifying relevant attributes, matching them with user preferences, and ranking items according to their relevance. In HackMate-Ai, hackathon details and user preferences support searching and filtering opportunities according to relevant criteria. Ranked search helps users explore suitable listings, while the skill-gap advisor identifies areas for improvement. The main advantage of this approach is that recommendations can be based on individual preferences without depending on ratings from other users. However, the effectiveness of this method depends on the quality and completeness of the available hackathon descriptions and user profiles.

3.1.1Existing System Architecture

Hackathon discovery systems need proper techniques for representing hackathon details and creating user profiles, and strategies for comparing user preferences with available opportunities.

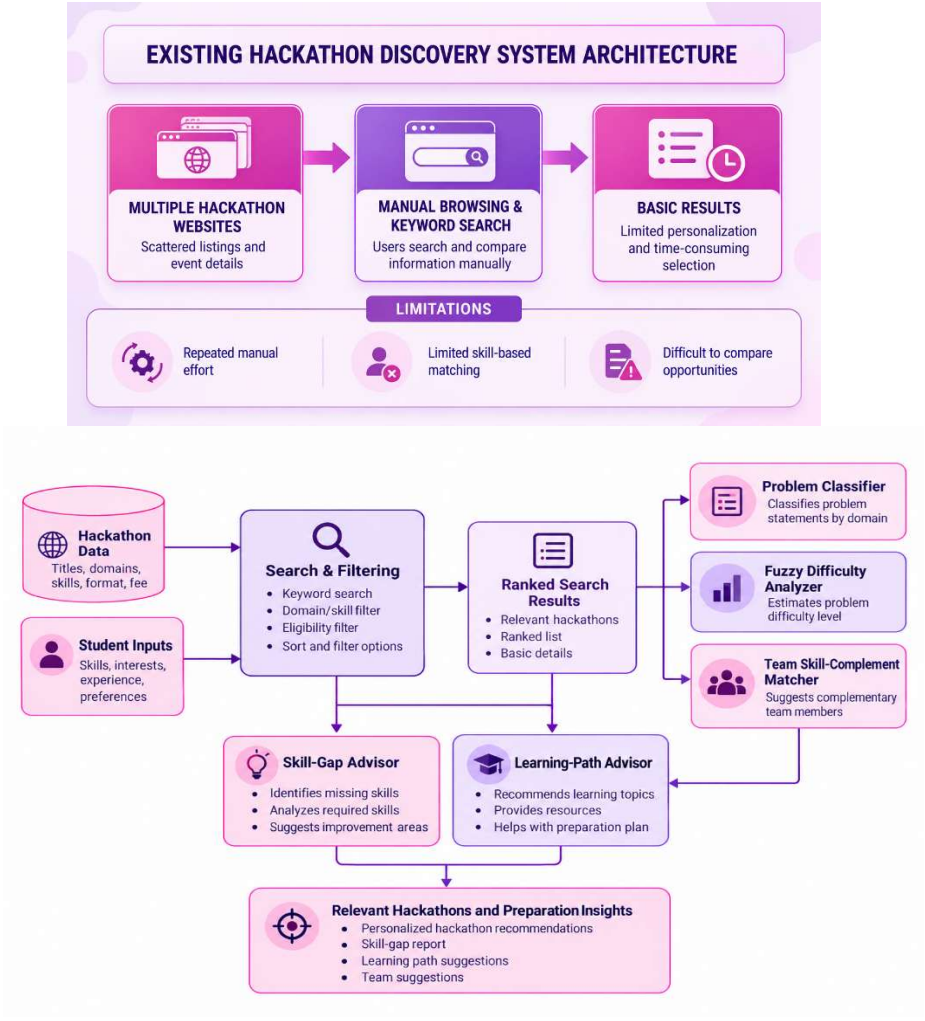


Fig. 3.2 Existing system architecture of HackMate-Ai

The high-level architecture of an existing hackathon discovery system is depicted in Figure 3.1. Its process is performed in three steps:

1. Hackathon Data Collector: Collects hackathon information from different websites and platforms, including event descriptions, required skills, participation formats, and registration details.

2. Basic Search and Filtering: Applies keyword-based searches and predefined filters to identify hackathons. However, basic search methods may not consider the user's complete skill profile or provide intelligent recommendations.

3. Search Results Display: Displays the matching hackathon listings to users. However, users may need to compare opportunities manually, identify missing skills independently, and determine suitable preparation strategies.

3.1.2Proposed System Architecture

The previous section discussed the limitations of existing hackathon discovery systems, which rely mainly on manual browsing and basic keyword searches. To provide more relevant opportunities and support student preparation, **HackMate-Ai** combines search, filtering, ranking, and AI-inspired techniques into an integrated web-based platform. The proposed architecture includes hackathon discovery, skill-gap analysis, learning-path recommendations, problem classification, fuzzy difficulty analysis, and team skill-complement matching. These modules work together to help students identify suitable hackathons, understand skill requirements, and prepare effectively. The proposed system architecture is shown in Figure 3.3.

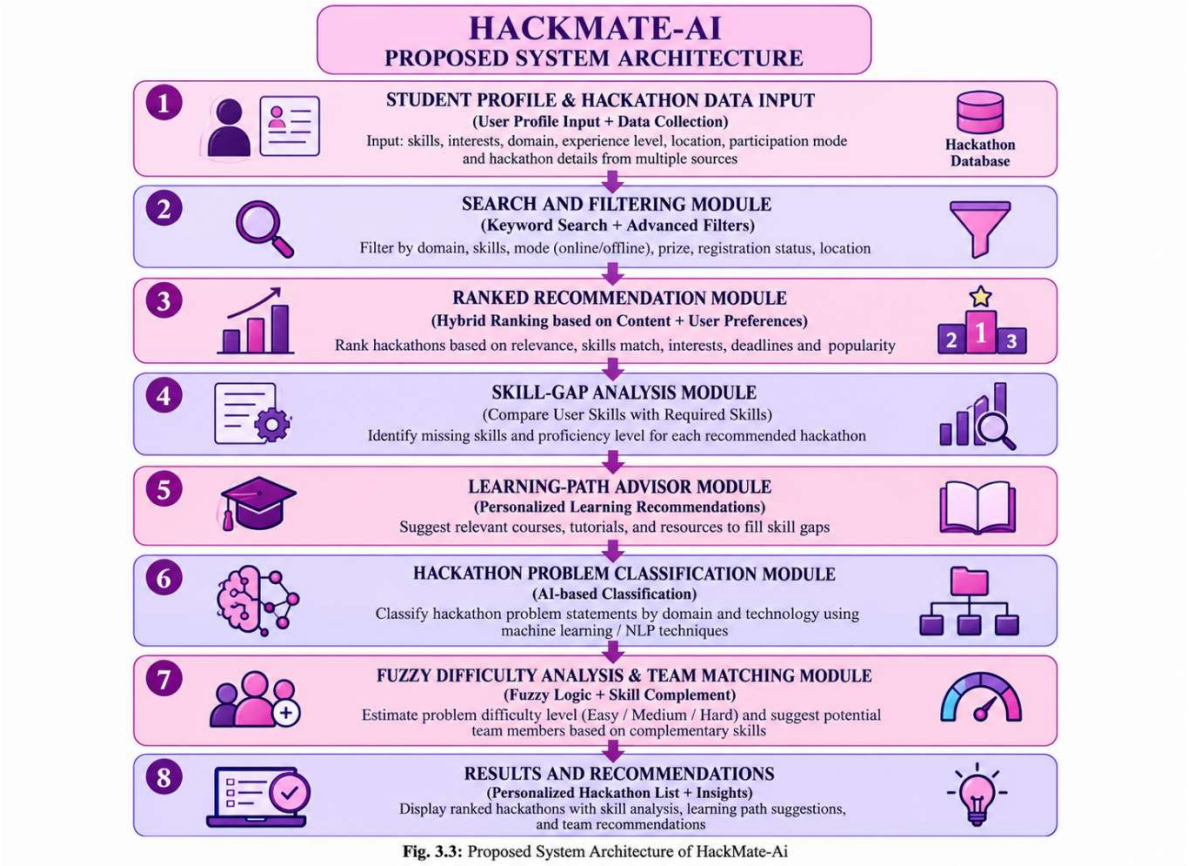


Fig. 3.3: Proposed System Architecture of HackMate-Ai

Fig. 3.3 Proposed system architecture Hackm

3.2.1Hybrid Recommendation and AI Techniques

Depending on user preferences and hackathon data characteristics, different combinations of AI and recommendation techniques can produce different results. HackMate-Ai integrates search, filtering, ranking, and AI-inspired methods to support hackathon discovery and student preparation. The following hybridization techniques are relevant to this domain.

3.2.2 Weighted Hybrid: This technique combines scores from multiple criteria using assigned weights. In HackMate-Ai, user interests, technical skills, and hackathon relevance can support opportunity ranking. Its advantage is that multiple criteria are considered together, while its limitation is the need for suitable weights.

3.2.4 Switching Hybrid: This technique selects a suitable method according to the current situation. In HackMate-Ai, users can apply different search terms and filters according to their preferences. It provides flexibility, but its effectiveness depends on appropriate selection criteria.

3.2.5 Mixed Hybrid: This technique combines multiple ranked lists into a single list. In hackathon discovery, results may be organized according to relevance and user preferences. It improves result organization, but requires consistent ranking criteria.

3.2.5 Feature Combination Hybrid: This method combines features from different information sources before processing. Hackathon domains, required skills, experience levels, and student interests can be considered together to improve matching. However, it depends on complete and accurate input data.

3.2.6 Feature Augmentation Hybrid: This method generates additional features from existing information. In HackMate-Ai, skill-gap analysis and problem classification provide additional insights for opportunity selection. The quality of these insights depends on the available data and analysis methods.

3.2.7 Cascade Hybrid: This technique processes results through successive stages, with each stage refining the previous output. HackMate-Ai follows a workflow involving search, filtering, ranking, and skill-related analysis. However, errors in earlier stages may affect subsequent results.

3.2.8 Meta-Level Hybrid: This method uses the output or representation produced by one model as input to another model. In HackMate-Ai, problem classification and skill analysis can provide contextual information for further analysis. Its effectiveness depends on the quality and compatibility of the information passed between components.

These techniques demonstrate how multiple approaches can complement one another to improve information discovery. Their practical benefits depend on the selected methods, input data, and implementation.

3.2.9 Implementation Details

The implementation details and procedural algorithm of HackMate-Ai are presented in this section. The system is developed using Python and Streamlit and provides an interactive platform for hackathon discovery, search, filtering, ranking, and AI-inspired student assistance.

3.2.10 Algorithm: HackMate-Ai Hackathon Discovery Pipeline

Input: Student profile (U), hackathon dataset (H), and user preferences (P).

Output: Relevant hackathon listings and skill-related insights.

Step 1 (User Profile Input): Accept the user's skills, interests, experience level, and participation preferences.

Step 2 (Search and Filtering): Search available hackathons and apply the selected filters.

Step 3 (Ranked Search): Rank matching opportunities according to the implemented relevance criteria.

Step 4 (Skill-Gap Analysis): Identify potential missing skills by comparing user skills with opportunity requirements.

Step 5 (Learning-Path Assistance): Suggest relevant learning topics and preparation directions.

Step 6 (Problem Classification and Difficulty Analysis): Categorize problem statements using the neural-style classifier and estimate their difficulty using the fuzzy analyzer.

Step 7 (Team Skill Matching): Identify complementary technical skills among team members.

Step 8 (Display Results): Present matching hackathons, rankings, skill insights, learning suggestions, and team-matching results.

3.4Use Case Diagram / Activity Diagram

Each block in the HackMate-Ai workflow performs a dedicated role:

1. **User Input and Search Block:** Collects student preferences and searches available hackathons using relevant filters.
2. **AI Analysis Block:** Performs ranked search, skill-gap analysis, learning-path suggestions, problem classification, and fuzzy difficulty analysis.
3. **Recommendation and Output Block:** Matches complementary team skills and displays relevant hackathons and personalized insights.

3.5 Output and Dataset

Experiments are conducted using hackathon records and student profile inputs to demonstrate the search, filtering, ranking, and AI-based features of HackMate-Ai. The dataset and inputs used by the system are summarized in Table 3.1.

Dataset / Input	Features	Purpose
Hackathon Dataset	Domain, skills, format, fee	Search and filtering
Student Profile	Skills, interests, experience	Personalized search
Problem Statements	Text descriptions	Classification and difficulty analysis
Team Skills	Individual technical skills	Team skill matching

Table 3.1: Sample Dataset Used in HackMate-Ai

3.6 Hardware and Software Specifications

HackMate-Ai is developed and executed using a computer system with suitable hardware and software configurations. The hardware specifications are presented in Table 3.2, while the software requirements are listed in Table 3.3.

Table 3.2: Hardware Details

Component	Specification
Processor	Intel Core i5 or equivalent
HDD / SSD	256 GB SSD
RAM	8 GB or higher

Table 3.3: Software Details

Category	Specification
Operating System	Windows 10/11
Programming Language	Python
AI & ML Libraries	NumPy, pandas, scikit-learn
Web Framework	Streamlit
Development Environment	Visual Studio Code
Version Control	Git and GitHub

Chapter 4

Performance and Results

4.1 Real-Time Hackathon Search and Recommendations

Figure 4.1 shows the HackMate-Ai web application during hackathon discovery. The dashboard displays the student profile, selected skills and interests, participation preferences, and experience level. It also shows the number of loaded hackathon listings and the Smart Hackathon Finder search bar. Each listing provides event details, required skills, skill-match information, and estimated difficulty, helping students explore suitable opportunities.

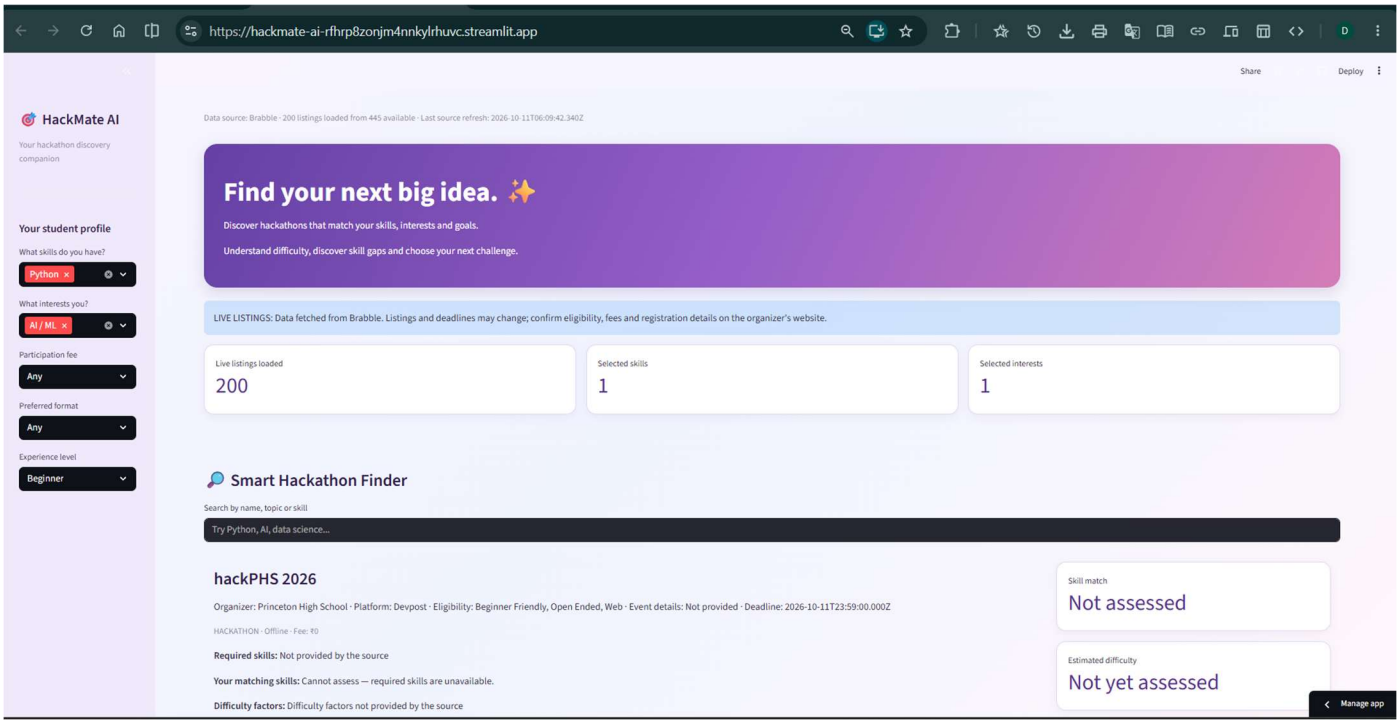


Figure 4.1: HackMate-Ai Dashboard for Hackathon Search and Recommendations

4.2 Detailed Hackathon Listings and Skill Analysis

Figure 4.2 presents the hackathon listings and analysis interface of HackMate-Ai. The Smart Hackathon Finder displays search results based on the user's selected skills, interests, and search keywords. Each listing provides details such as the organizer, platform, eligibility, participation format, registration fee, and deadline. The skill-match and estimated-difficulty panels provide additional insights when sufficient information is available, helping students evaluate hackathon opportunities.

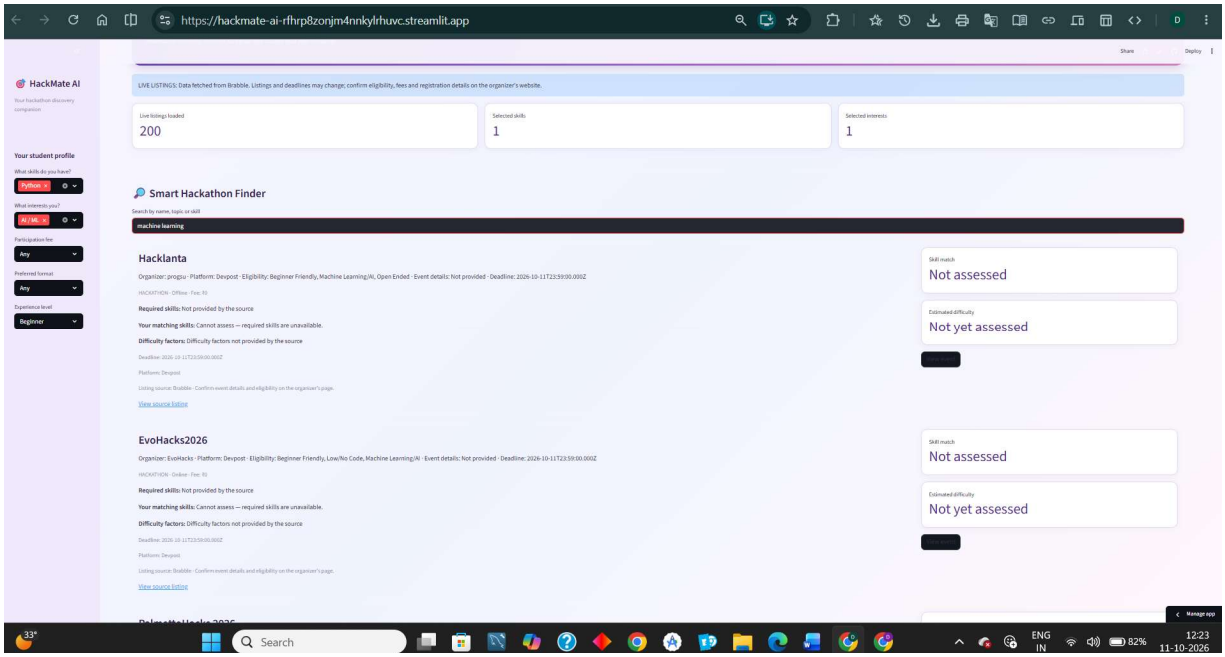


Fig. 4.2: Detailed Hackathon Listings with Skill-Match and Difficulty Analysis

4.3 Neural Network Problem Classification

Figure 4.3 illustrates the Neural Network Problem Classifier of HackMate-Ai. The user enters a hackathon problem statement, and the system predicts its technical category. In the displayed example, the input "build a shopping website" is classified under Web Development, with the recognized words shown below the result. This feature helps students identify the technical domain of a hackathon challenge and explore opportunities related to their interests.

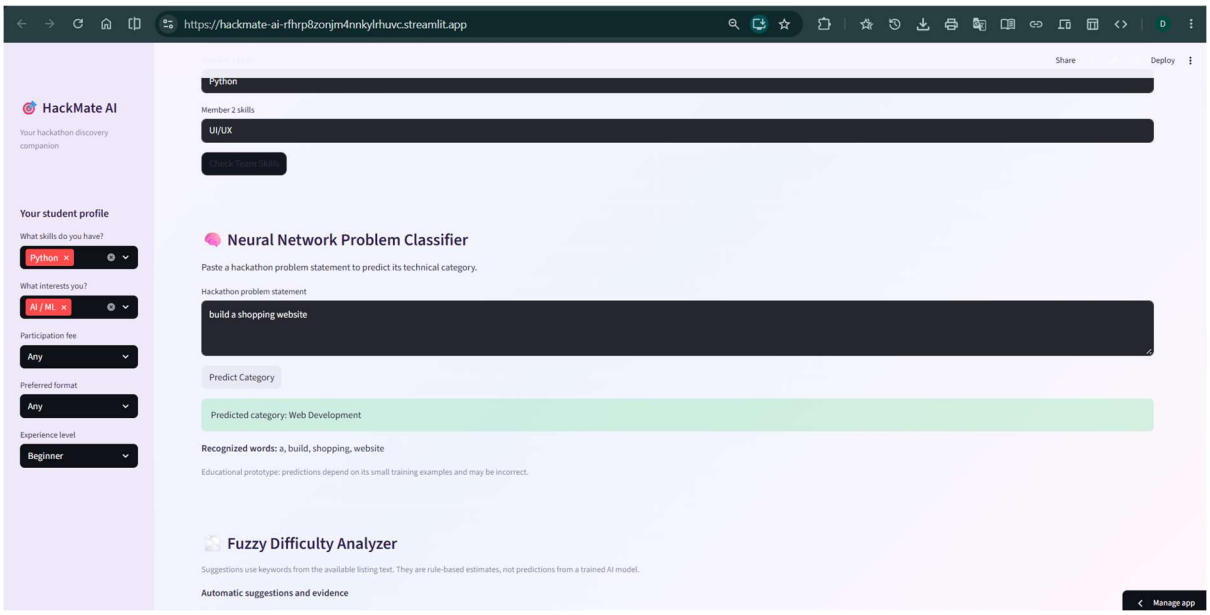


Fig. 4.3: Neural Network Problem Classification in HackMate-Ai

4.5 Team Skill-Complement Matching

Figure 4.5 illustrates the Hackathon Team Skill-Complement Matcher in HackMate-Ai. The module compares the required skills with the skills available among team members. In the displayed example, Python and UI/UX are already covered, while Machine Learning is identified as a missing skill. The system recommends finding a teammate with Machine Learning skills to improve the team's skill coverage.

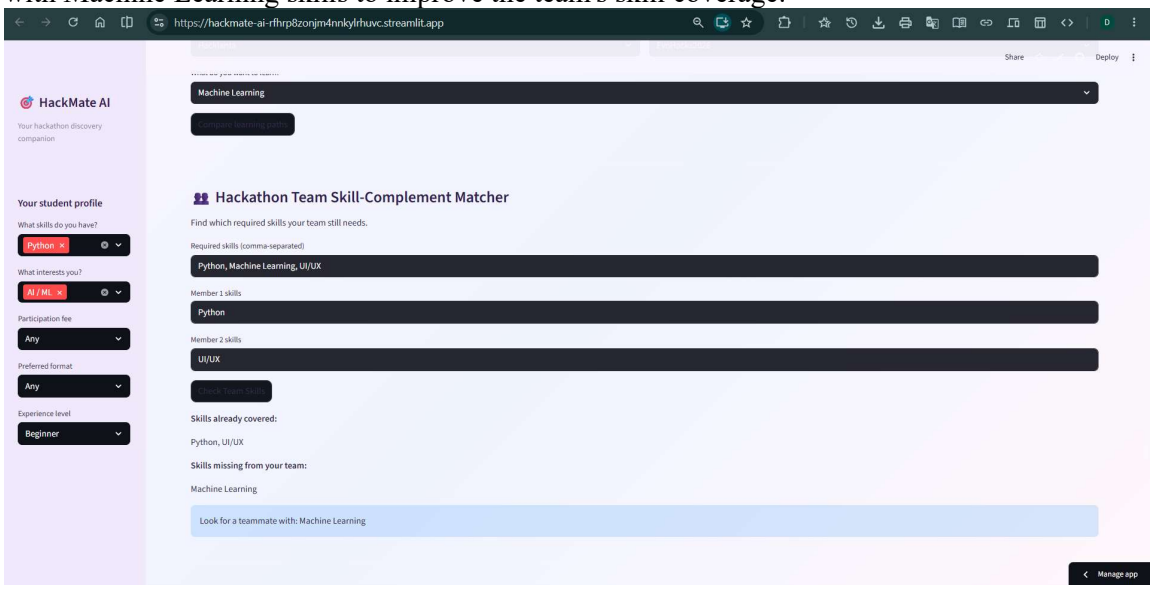


Fig. 4.5: Team Skill-Complement Matching in HackMate-Ai

4.6 Hackathon Learning Path Advisor

Figure 4.6 shows the Hackathon Learning Path Advisor of HackMate-Ai. It compares two hackathons based on the selected learning goal. In this example, Hacklanta and EvoHacks2026 are compared for Machine Learning. The system identifies relevant topics and suggests learning directions, helping students prepare for suitable hackathon challenges.

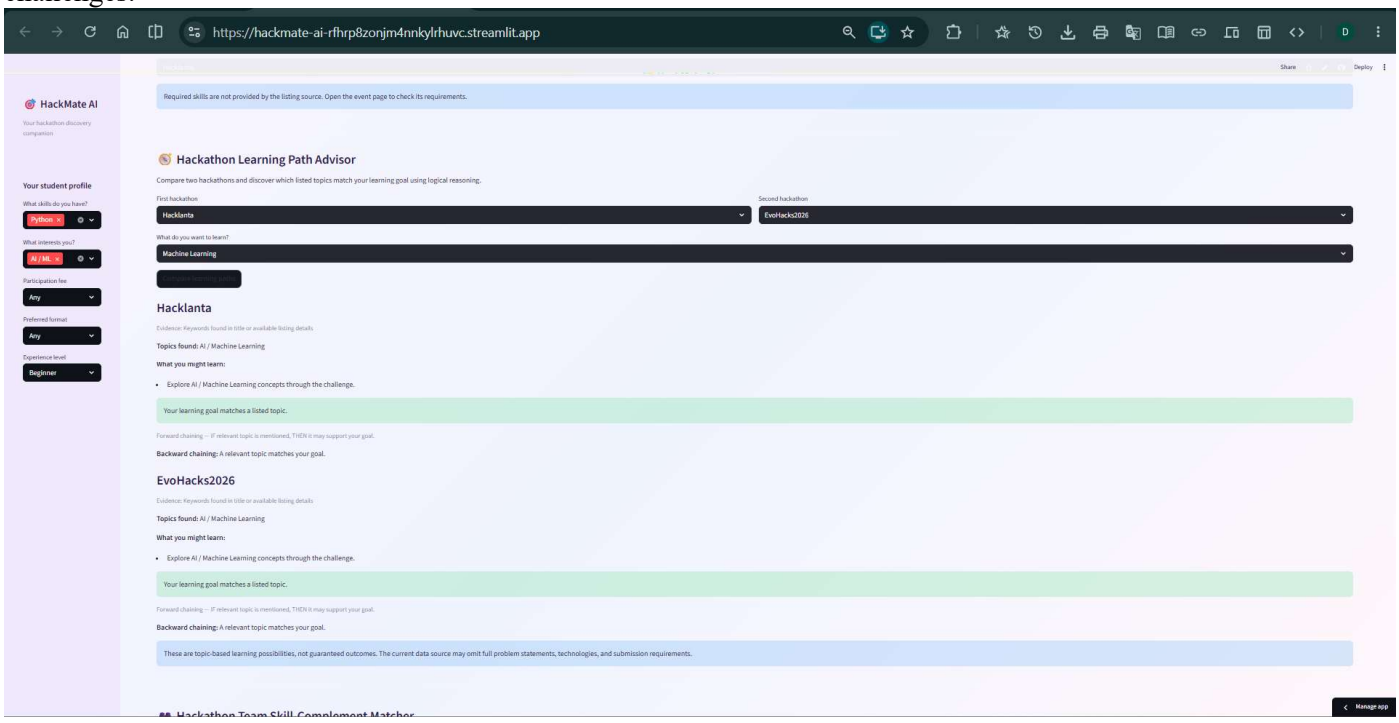


Fig. 4.6: Hackathon Learning Path Advisor in HackMate-Ai

4.7 Fuzzy Difficulty Analyzer

Figure 4.7 illustrates the Fuzzy Difficulty Analyzer of HackMate-Ai. The module evaluates factors such as technical complexity, prerequisites, expected deliverables, and time pressure to estimate the difficulty of a hackathon challenge. In the displayed example, the estimated difficulty is Moderate, with an average factor score of 3.50 out of 5. The fuzzy membership values and identified reasons help students understand the factors influencing the estimated difficulty.

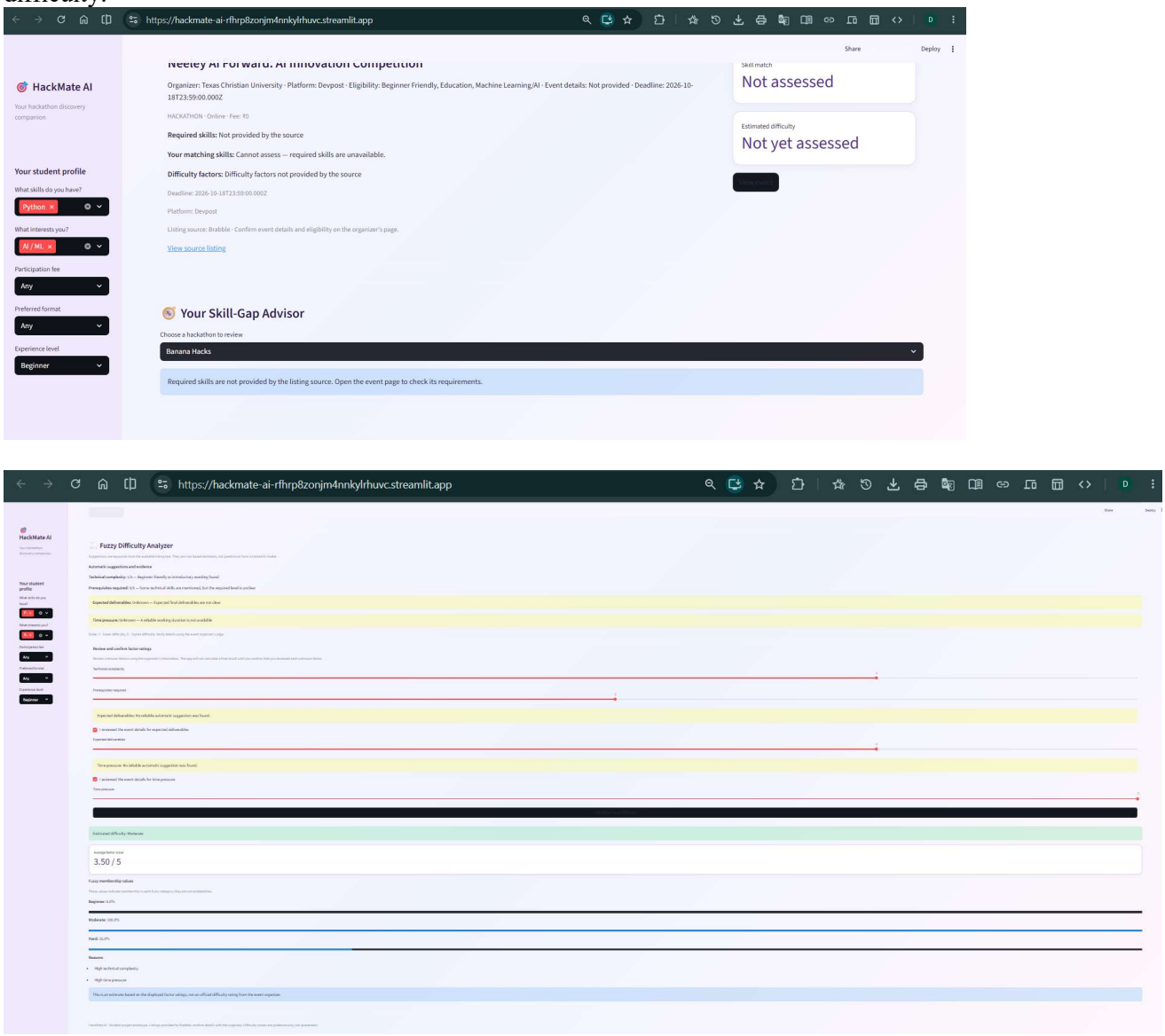


Fig. 4.7: Fuzzy Difficulty Analysis in HackMate-Ai

4.8 Comparative Results and Feature Verification

Table 4.1 summarizes the main feature verification results of HackMate-Ai. The system is evaluated using sample hackathon listings, student profiles, and problem statements to check its search, recommendation, classification, and analysis features.

Table 4.1: Feature Verification of HackMate-Ai

Feature	Expected Outcome
Hackathon Search	Displays relevant listings using search and filters
Problem Classifier	Predicts a technical category
Learning Path Advisor	Identifies topics matching the learning goal
Team Skill Matcher	Identifies missing team skills
Fuzzy Difficulty Analyzer	Estimates difficulty using selected factors

Table 4.2 compares conventional hackathon discovery with the proposed HackMate-Ai platform.

Table 4.2: Comparison of Existing and Proposed Systems

Parameter	Existing System	Proposed HackMate-Ai
Hackathon Discovery	Manual browsing	Search and filtering
Recommendations	Limited personalization	Skill- and interest-based exploration
Problem Analysis	Manual interpretation	Category classification
Difficulty Assessment	Manual estimation	Fuzzy difficulty analysis
Team Formation	Manual skill comparison	Team skill-complement matching
Learning Support	Separate resources	Integrated learning-path advisor

Chapter 5

Applications

HackMate-Ai provides a centralized platform for hackathon discovery, student skill development, problem analysis, and team collaboration. It combines search, recommendation, and AI-based analysis features to help students identify suitable hackathons and prepare for participation. The major social, educational, and technical applications are described below.

5.1 Social and Educational Applications

5.1.1 Hackathon Awareness and Discovery: HackMate-Ai helps students discover hackathons based on their skills, interests, experience levels, participation format, and registration fees. It reduces the effort required for manual browsing.

5.1.2 Skill Development and Learning Support: The Skill-Gap Advisor helps identify skills that may need improvement, while the Learning Path Advisor suggests relevant topics to support student preparation.

5.1.3 Informed Hackathon Participation: The platform displays event details and estimated difficulty, helping students evaluate opportunities according to their abilities and learning goals.

5.2 Technical and Institutional Applications

5.2.1 Intelligent Hackathon Search: Search and filtering features allow users to explore available hackathons using keywords and participation preferences.

5.2.2 Problem Classification: The Neural Network Problem Classifier predicts the technical category of a problem statement, helping students understand the domain of a challenge.

5.2.3 Fuzzy Difficulty Analysis: The Fuzzy Difficulty Analyzer estimates challenge difficulty using factors such as technical complexity, prerequisites, expected deliverables, and time pressure.

5.2.4 Team Skill Matching: The Team Skill-Complement Matcher compares required skills with team members' skills and identifies missing expertise that may be needed.

5.2.5 Learning Path Recommendation: The Learning Path Advisor compares selected hackathons against a learning goal and identifies relevant topics, supporting better preparation and learning decisions.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

In this project, **HackMate-Ai** was developed as an intelligent hackathon discovery and recommendation platform using Python and Streamlit. The system integrates search, filtering, ranked results, skill-gap analysis, learning-path recommendations, problem classification, fuzzy difficulty analysis, and team skill matching. These features help students discover relevant hackathons, understand challenge requirements, identify missing skills, and prepare effectively. The platform combines AI and Soft Computing concepts to support informed decision-making and improve the hackathon discovery experience.

6.2 Future Scope

- **Integration with Live Hackathon Sources:** Connect the platform to external hackathon websites and APIs to provide updated event information, deadlines, and registration links.
- **Advanced AI Recommendations:** Improve recommendation quality using larger datasets and more advanced machine learning techniques based on student skills and interests.
- **Personalized Learning Resources:** Integrate tutorials, courses, and practice materials to help students develop the skills required for selected hackathons.
- **Team Collaboration Features:** Extend team matching with user profiles, team creation, collaboration tools, and improved skill-complement recommendations.
- **Enhanced User Interface:** Develop additional features such as user accounts, saved hackathons, personalized dashboards, and notifications about upcoming events.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

[2] L. A. Zadeh, "Fuzzy Sets," *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.

[3] F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
[4] S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[5] J. MacQueen, "Some Methods for Classification and Analysis of Multivariate Observations," in *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967.

[6] R. E. Bellman, *Dynamic Programming*. Princeton University Press, 1957.

[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[8] Python Software Foundation, "Python Documentation." [Online]. Available: https://docs.python.org/3/

[9] Streamlit, "Streamlit Documentation." [Online]. Available: https://docs.streamlit.io/

[10] scikit-learn developers, "scikit-learn User Guide." [Online]. Available: https://scikit-learn.org/stable/user_guide.html

Acknowledgement

We would like to express our sincere gratitude to our mini project guide, **Prof. Nirosha Uppu**, for her valuable guidance, encouragement, and support throughout the development of our project, **HackMate-Ai**. Her guidance helped us apply the knowledge acquired during the semester and understand the practical applications of Artificial Intelligence and Soft Computing.

We are thankful to **Dr. Shankar M. Patil**, Head of the Department of Artificial Intelligence and Data Science, for providing us with the opportunity to undertake this mini project and explore new concepts and technologies.

We would also like to thank our mini project coordinator, **Prof. Nirosha Uppu**, and all the faculty members for their continuous encouragement, valuable suggestions, and support during the completion of this project.

Finally, we express our sincere gratitude to our Principal, **Dr. Sunil Chavan**, for providing the necessary facilities and support to carry out this mini project successfully.

Divya Dhanawade